ESCUELA SUPERIOR POLITÉCNICA DEL LITORAL
FACULTAD DE INGENIERÍA EN ELECTRICIDAD Y COMPUTACIÓN

AÑO:	2024	**PERIODO:**	SEGUNDO TÉRMINO
MATERIA:	SISTEMA DE BASES DE DATOS	**PROFESORES:**	IRENE CHEUNG, FRANK MALO
EVALUACIÓN:	SEGUNDA	**FECHA:**	28/01/2025

NOMBRE: _____ **MATRÍCULA:** _____

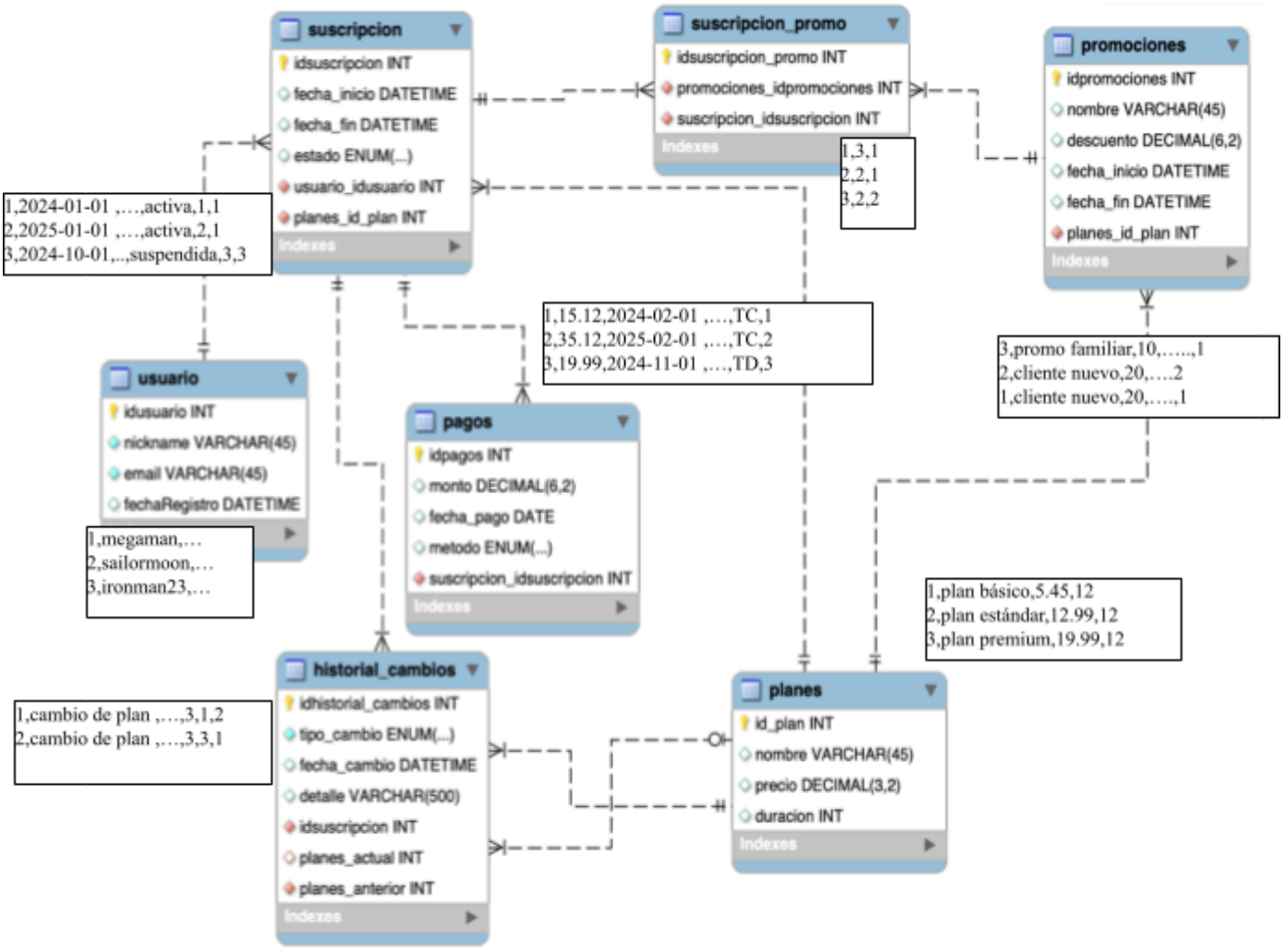


Imagen 1. Base de datos de NECFLISDB

<u>Sección 1 (25%)</u> DDL Y DML: Escriba las sentencias DDL para:

- Crear la tabla **suscripción** (idsuscripcion, fecha_inicio, fecha_fin, estado, usuario_idusuario, planes_id_plan) considerando:
    - Idsuscripcion es la pk
    - usuario_idusuario y planes_id_plan son fk de usuario y planes
    - estado es un enumerado que contiene “activo”,”suspendido”,”cancelado”
    - todos los campos son obligatorios
    - poner una restricción que la fecha fin debe ser posterior a la fecha _inicio

```
CREATE TABLE Suscripcion (
  ID_Suscripcion INT PRIMARY KEY AUTO_INCREMENT,
  Fecha_Inicio DATETIME NOT NULL,
  Fecha_Fin DATETIME NOT NULL,
  Estado ENUM('activo', 'suspendido', 'cancelado') NOT NULL,
  Usuario_IDUsuario INT NOT NULL,
  Planes_ID_Plan INT NOT NULL,
  CONSTRAINT fk_usuario FOREIGN KEY (Usuario_IDUsuario) REFERENCES Usuarios(ID_Usuario),
  CONSTRAINT fk_plan FOREIGN KEY (Planes_ID_Plan) REFERENCES Planes(ID_Plan),
  CONSTRAINT chk_fecha CHECK (Fecha_Fin > Fecha_Inicio) );
```

- Cambia el plan de la suscripción del usuario "Hulk" al “plan premium" y registra este cambio en el Historial_Cambios.

```
UPDATE Suscripciones
SET ID_Plan = (SELECT ID_Plan FROM Planes WHERE Nombre_Plan = 'Premium')
WHERE ID_Usuario = (SELECT ID_Usuario FROM Usuarios WHERE Nombre = 'Hulk');

INSERT INTO Historial_Cambios (ID_Suscripcion, Tipo_Cambio, Fecha_Cambio)
VALUES (
  (SELECT s.ID_Suscripcion
   FROM Suscripciones s
   JOIN Usuarios u ON s.ID_Usuario = u.ID_Usuario
   WHERE u.Nombre = 'Hulk'),
  'Cambio de plan',
  NOW()
);
```

- Añadir un campo **estado** a la tabla **pagos** que solo permita los textos “pendiente” y “confirmado”. Además, el campo **monto** de la misma tabla debe ser obligatorio y mayor a cero.

```
ALTER TABLE Pagos
ADD COLUMN Estado ENUM('pendiente', 'confirmado') NOT NULL DEFAULT 'pendiente';
ALTER TABLE Pagos
ADD CONSTRAINT chk_monto CHECK (Monto > 0);
```

<u>Sección 2 (30%)</u> Consultas SQL: Tomando como referencia la base resultante de la Sección 1, realice las siguientes consultas:

a) Calcular el ingreso total generado por plan. Mostrar el nombre del plan, el total generado en dólares y el total de suscriptores ordenados de forma descendente por total de suscriptores.

```
SELECT
  p.Nombre_Plan,
  COUNT(s.ID_Suscripcion) AS Total_Suscriptores,
  SUM(p.Precio - IFNULL(pr.Descuento, 0)) AS Ingreso_Total
FROM Suscripciones s
JOIN Planes p ON s.ID_Plan = p.ID_Plan
LEFT JOIN Suscripciones_Promociones sp ON s.ID_Suscripcion = sp.ID_Suscripcion
LEFT JOIN Promociones pr ON sp.ID_Promocion = pr.ID_Promocion
GROUP BY p.Nombre_Plan
ORDER BY Total_Suscriptores DESC;
```

b) Determinar los usuarios que han cambiado de plan más de una vez.

```
SELECT u.ID_Usuario, u.Nombre, COUNT(h.ID_Cambio) AS Total_Cambios
FROM Historial_Cambios h
JOIN Suscripciones s ON h.ID_Suscripcion = s.ID_Suscripcion
JOIN Usuarios u ON s.ID_Usuario = u.ID_Usuario
WHERE h.Tipo_Cambio = 'Cambio de plan'
GROUP BY u.ID_Usuario, u.Nombre
HAVING COUNT(h.ID_Cambio) > 1;
```

c) Muestra las suscripciones que tienen y no tienen promociones. Muestra la suscripción y el nombre de la promoción.

```
SELECT
        s.id_suscripcion, (SELECT nombre FROM PROMOCIONES P WHERE
        sp.promociones_idpromociones = P.idpromociones)
FROM Suscripciones s
LEFT JOIN Suscripciones_Promociones sp ON s.ID_Suscripcion = sp.ID_Suscripcion
```

d) Encuentra las promociones que se han aplicado a al menos una suscripción con un plan de menor costo que cualquier otro plan.

```
SELECT DISTINCT pr.ID_Promocion, pr.Nombre_Promocion, pr.Descuento
FROM Suscripciones_Promociones sp
JOIN Suscripciones s ON sp.ID_Suscripcion = s.ID_Suscripcion
JOIN Planes p ON s.ID_Plan = p.ID_Plan
JOIN Promociones pr ON sp.ID_Promocion = pr.ID_Promocion
WHERE p.Precio = (SELECT MIN(Precio) FROM Planes);
```

e) Cotizar el plan “VIP” que tiene una duración en meses considerando las promociones que tengan y su duración (usualmente las promociones tienen una duración mensual). Mostrar el nombre del plan, precio total, total de descuentos si tiene por promociones y el total que se pagaría.

```
SELECT
   p.Nombre_Plan,
   (p.Precio * p.Duracion) AS Precio_Total,
   SUM(pr.Descuento *  TIMESTAMPDIFF(MONTH, pr.Fecha_Inicio, pr.Fecha_Fin)) AS Total_Descuentos,
   (p.Precio * p.Duracion) - Total_Descuentos AS Total_Pagar
FROM Planes p
JOIN Promociones pr ON p.id_plan = pr.planes_id_plan
WHERE p.Nombre_Plan = 'VIP'
GROUP BY p.Nombre_Plan;
```

Sección 3 (35%) Objetos avanzados Considerando el esquema de base de datos de NECFLISDB presentado en la Imagen 1 escribir las sentencias correspondientes para lo siguiente:

a) Crear un disparador llamado **tgr_historial** que genere el historial de suscripción y/o cambio de plan de un usuario. Considerar en el campo **tipo_cambio** de la tabla **historial_cambios** colocar

‘cambio de plan’ o ‘nueva suscripción’. Hay que recordar que puede ser el primer historial para un cliente, o puede ser un cambio de suscripción.

```
delimiter %
create trigger tgr_historial after insert on suscripcion
for each row
   insert into historia_cambios (tipo_cambio, fecha_cambio, detalle, idsuscripcion ,planes_actual, planes_anterior)
   values('cambio de plan', now(), 'cambio de plan', new.idsuscripcion, new.planes_id, null);
%
delimiter ;

delimiter %
create trigger tgr_historial after update on suscripcion
for each row
   if old.planes_i <>  new.planes_id then
      insert into historia_cambios (tipo_cambio, fecha_cambio, detalle, idsuscripcion ,planes_actual, planes_anterior)
      values('cambio de plan', now(), 'cambio de plan', new.idsuscripcion, old.planes_id, new.planes_id);
   end if;
%
delimiter ;
```

b) Crear un procedimiento llamado **sp_upgrade_plan** en el cual un usuario es actualizado a un nuevo plan con un precio superior. Los parámetros de entrada deben ser:

- Id del nuevo plan
- Id del usuario
- Detalle del cambio
- Fecha de inicio del nuevo plan
- Fecha de fin del nuevo plan
- Datos del primer pago
- Parámetro de salida tipo bool indicando el resultado de la operación

Considerar lo siguiente:

- Validar que el precio del nuevo plan sea superior al del anterior plan.
- Usar transacciones y validar excepciones.
- Puede utilizar la función LAST_INSERT_ID() para obtener el id del registro recientemente insertado

```
delimiter %
create procedure sp_upgrade_plan (in id_nuevo int, in id_usuario int,
   in p_detalle varchar(500), in p_fecha_inicio datetime,
   in p_fecha_fin datetime, in p_monto decimal(6,2),
   in p_fecha_pago date, in p_metodo int, out salida bool)
begin

   declare exit handler for sqlexception
   begin
      rollback;
      set salida = false;
   end;

   start transaction;

   select idsuscripcion, planes_id_plan into @suscripcion, @plananterior
   from suscripcion where usuario_idusuario = id_usuario;
```

```
    insert into historia_cambios(tipo_cambio, fecha_cambio, detalle, idsuscripcion, planes_actual, planes_anterior)
    values ('cambio_plan', now(), p_detalle, @suscripcion, id_nuevo,  @plananterior );

    update suscripcion as s
    set planes_id_plan = id_nuevo
    where idsuscripcion = @suscripcion;

    insert into pagos(monto, fecha_pago, metodo, suscripcion_idsuscripcion)
    values (p_monto, p_fecha_pago, p_metodo, @suscripcion);

    set @valor_anterior= (select precio from planes where id_plan = @plananterior );
    set @valor_nuevo= (select precio from planes where id_plan = id_nuevo );

    if @valor_nuevo <= @valor_anterior then
       rollback;
    end if;

    commit;
    set salida = true;

end;
%
delimiter ;
```

c) Crear un reporte que presente los datos de los usuarios que tengan planes activos y tengan pagos atrasados. Presentar los datos completos del usuario, nombre del plan, meses de atraso y total de dinero adeudado. Tip: puede utilizar la función TIMESTAMPDIFF(MONTH, '$YYYY_i$-MM_i-DD_i', '$YYYY_f$-MM_f-DD_f') para obtener el total de meses entre 2 fechas.

```
create view v_reporte as
select distinct u.*, TIMESTAMPDIFF(MONTH, now(), pagos_maximos.fecha) as meses_tarde
from usuario u
inner join suscripcion s on u.idusuario=s.usuario_idusuario
inner join (
   select suscripcion_idsuscripcion, max(fecha_pago) as fecha
   from pagos
   group by suscripcion_idsuscripcion
) pagos_maximos
on s.idsuscripcion = pagos_maximos.suscripcion_idsuscripcion
```

Sección 4 (10%) Índices y seguridades

a) Crear un usuario analista@'localhost' y concederle los siguientes permisos.
CREATE USER analista@'localhost';

1. Permiso de lectura a la vista del tema 3.c
GRANT SELECT ON NECFLISDB.v_reporte TO analista@'localhost';
FLUSH PRIVILEGES;

2. Permiso para ejecutar el procedimiento del literal 3.a
GRANT execute ON NECFLISDB.sp_upgrade_plan TO analista@'localhost';
FLUSH PRIVILEGES;

3. Asumiendo que la tabla de usuario tiene un campo password, encripte con md5 las contraseñas de los usuarios actuales. Tip: la función md5(*pass*) genera el código hash del password.

update usuario set password = md5(password)

b) Optimizar la siguiente consulta por medio de un índice:

```
select nickname, email
from usuario inner join suscripcion on idusuario=usuario_idusuario
where fecha_inicio >= <parametro> and estado = <parametro>
```

Create index ix_usuario on usuario (fecha_inicio, estado);

c) Para el índice anterior es preferible usar un algoritmo de hash o de btree ?

btree